



Escuela de Ingeniería en Computadores

CE 1108 – Compiladores e intérpretes

Proyecto - Avance 1

Definición de Sintaxis de Lenguaje

Profesor:

Jason Leitón Jiménez

Estudiantes:

David García Cruz - 2024115575

Tamara Vanessa Cajiao Molina - 2024143333

Luis Felipe Chaves Mena - 2024192941

Santiago Robles Obando - 2022207100

Fecha de Entrega:

II Semestre del 2026

11 de setiembre

Introducción

Los compiladores e intérpretes actúan como el puente de traducción entre las abstracciones de alto nivel utilizadas por los programadores y el lenguaje de máquina ejecutable por la arquitectura del procesador. Este proyecto consiste en el diseño de un nuevo lenguaje de programación, lo cual requiere una definición cuidadosa y sistemática de su sintaxis y semántica, estableciendo así las reglas léxicas y gramaticales que determinan la validez de cada instrucción escrita en el código fuente.

En el presente documento se presenta la definición formal de la sintaxis para nuestro lenguaje de programación original. Se detalla la estructura léxica del lenguaje mediante la categorización de sus tokens, palabras reservadas, tipos de datos, así como la gramática para la declaración de variables, rutinas y estructuras de control iterativas y condicionales. Adicionalmente, se establecen las convenciones de ámbito de bloque, el manejo de comentarios y se ilustra la aplicabilidad del lenguaje mediante algoritmos desarrollados como casos de prueba.

Desarrollo

Estructura de lenguaje de programación **Piper**:

1. Tabla de tokens

Categoría	Ejemplos
Palabras reservadas	create, when, routine, bool, B8, uB8, uB16, B16, B32, uB32, loop, alwhen, break, output, fallback, none, true, false
Operadores Aritméticos	+, -, *, /, ^, ++, --, %
Operadores de Asignación	store
Operadores Lógicos	=, <, >, !=, >=, <=, &&, , and, xor, or, ~
Delimitadores	{, }, ., ~~, [,], &, , ->, ' ', " , ***
Identificadores	Nombres de variables y funciones
Constantes	Números enteros, cadenas

1.1. Explicación de Palabras Reservadas

Palabra Reservada	Explicación
create	Declaración o creación de una variable.
when /fallback	Estructuras condicionales análogas a if/else
routine	Declaración de una función.
loop	Estructura iterativa análoga a for .
alwhen	Estructura iterativa análoga a while .
break	Termina el ciclo alwhen en cierto punto.
output	Devuelve el resultado de una función análoga a return .
store	Principal operador de asignación.
Tipos de datos	
uB8	Entero sin signo de 8 bits.
B8	Entero con signo de 8 bits.
uB16	Entero sin signo de 16 bits.
B16	Entero con signo de 16 bits.
uB32	Entero sin signo de 32 bits.
B32	Entero con signo de 32 bits.
bool	Valor booleano: true o false

2. La sintaxis de variables, funciones y estructuras de control.

2.1. Estructura de creación de variables

Se utiliza la palabra reservada **create** para declarar variables, ya sea de tipo simple (enteros, booleanos, etc.) o de tipo compuestos (listas o matrices).

La sintaxis general es:

```
create <data_type> <variable_name>.
```

Para arreglos o matrices, se definen las dimensiones de la matriz entre los delimitadores **|** y se separan con **&**.

```
create <data_type>|<row> & <columns>|<name>.
```

2.2. Estructura de función

Para crear una función (o rutina) se utiliza la palabra reservada **routine** seguida del tipo de dato del valor a retornar. Permite, mediante llaves, definir el bloque de código a ejecutar y devolver un valor mediante **output**.

```
routine <return_type> <name>[<param_data_type> <param_name>]{<content.>}.
```

2.3. Estructura de control condicionales

El lenguaje utiliza **when** como estructura condicional principal (equivalente al **if** equivalente en otros lenguajes). Además, se implementa **fallback** como bloque alternativo (equivalente al **else**). La expresión a evaluar que se utiliza para discriminar cual de los bloques de código a correr viene dada entre paréntesis cuadrados **[]**.

```
when [<condition>]{  
    <main_blk>  
} fallback {  
    <fallback_blk>  
}
```

El bloque **fallback** es opcional.

2.4. Estructura de control iterativa

2.4.1. tipo for

Existe la palabra reservada **loop**, con la cual se define una iteración controlada por un contador. Esta estructura requiere que se especifique una variable de control, un rango de inicio y fin (con **->**) y un paso a través de la evaluación de una expresión.

```
loop(<variable> & <begin> -> <end> & <step>){ <content.>. }
```

2.4.2. tipo while

Mediante la palabra reservada **alwhen** una iteración condicional, equivalente al **while** de otros lenguajes. El bloque se ejecuta repetidamente mientras la condición entre corchetes sea verdadera.

Para interrumpir la ejecución del ciclo de manera prematura, se implementa la palabra reservada **break**. Al encontrar esta instrucción, el compilador fuerza la salida inmediata del bloque **alwhen**.

```
alwhen[<condition>]{ <content.>
```

```
break. ~~Interrumpe el ciclo  
}
```

3. Definición de comentarios

Para tener una forma en la que hacer anotaciones a los bloques de código que se programen en este lenguaje, se decidió utilizar la combinación de caracteres `~~` para denotar el inicio de un comentario en el código. Se optó por dos `~` seguidos, pues la palabra reservada como operador lógico `~` es utilizada para denotar negación de cierta variable. Adicionalmente, para los comentarios en bloques, se declaró la combinación de caracteres `***`, esto pues el carácter `*` ya existe como palabra reservada como operador aritmético para multiplicación; además, es distinto a la declaración de comentario de una sola línea para hacer distinción de cuando no se debe compilar una o varias líneas. A continuación, un ejemplo de uso:

```
~~ Esto es un comentario antes de una suma  
x store y + 5  
*** Otro comentario,  
pero debajo de suma  
y es de varias líneas ***
```

4. Ámbito de bloque (cómo se delimita con , indentación, begin/end, etc.)

El lenguaje delimita los bloques mediante llaves `{ }`. Esto significa que una variable declarada dentro de un bloque sólo es accesible desde ese bloque y desde los bloques anidados en su interior, pero deja de existir en el momento en que el bloque se cierra con `}`. Además, cada línea/instrucción es delimitada con un punto `(.)`.

5. Información sobre el proceso de compilación y enlazado.

5.1. Invocación del compilador

La sintaxis general de comandos para procesos de compilación de `Piper` es:

```
pipoc <file> [-option]
```

Opción	Descripción
-o <archivo>	Especifica el nombre del archivo de salida. Si se omite, se usa el nombre del fuente con extensión <code>.bin</code> .
-c	Solo compila, no enlaza. Genera un archivo objeto (<code>.obj</code>), útil cuando el programa está dividido en varios <code>.rwt</code> .
-v	Modo detallado (<i>verbose</i>): imprime cada fase del proceso (léxico, sintáctico, semántico, generación de código).
-t	Imprime el árbol sintáctico (AST) generado en consola.

-I <directorio>	Agrega un directorio donde buscar los archivos referenciados por <code>bring ... from</code> .
-Werror	Trata los warnings como errores fatales.

5.2. Ejemplos de uso

Compilar un archivo y generar binario con nombre por defecto

```
pipoc programa.rwt
```

Compilar y especificar nombre de salida

```
pipoc programa.rwt -o output.bin
```

Compilar en modo detallado mostrando el AST

```
pipoc programa.rwt -v -t
```

Solo compilar sin enlazar (genera archivo objeto)

```
pipoc -c modulo.rwt -o module.obj
```

Agregar directorio de búsqueda para módulos importados

```
pipoc main.rwt -I ./modules
```

Compilar tratando los warnings como errores fatales

```
pipoc programa.rwt -Werror
```

5.3. Enlace de Múltiples Archivos

El lenguaje utiliza importación selectiva mediante `bring <symbol> from "<file>".`, que trae al ámbito del archivo actual únicamente la rutina indicada (no el archivo completo). Cada archivo `.rwt` se compila de manera independiente hasta un archivo objeto (`.obj`) que contiene:

- Los símbolos que define y exporta (toda `routine` de nivel superior).
- Una tabla de referencias externas pendientes (los símbolos traídos con `bring` que aún no tienen dirección resuelta).

El proceso de enlace ocurre así:

1. Se compila cada archivo fuente por separado (fase de análisis léxico, sintáctico y semántico, más generación de código objeto).
2. El enlazador recibe todos los `.obj` del programa y construye una tabla global de símbolos combinando los exportados de cada uno.

3. Por cada referencia pendiente, busca el símbolo en la tabla global y sustituye la referencia por la dirección real de la rutina.
4. Si todas las referencias se resuelven, genera el binario final; si no, reporta error (ver 5.3).

Ejemplo (tomado del Ejercicio 3): `main.rwt` hace `bring derive_key from "keys.rwt".` y `bring transform from "cipher.rwt".`; el compilador genera `keys.obj`, `cipher.obj` y `main.obj` por separado, y el enlazador resuelve `derive_key` y `transform` contra los símbolos exportados de `keys.obj` y `cipher.obj` respectivamente.

5.4. Resolución de Errores entre Archivos

En esta sección se especifica cómo se compila un programa escrito en su lenguaje, desde el código fuente hasta el binario ejecutable, y cómo se pueden combinar múltiples archivos fuente en un mismo programa. Antes de ser convertido a un binario ejecutable, el código fuente pasa por las siguientes fases:

1. **Análisis léxico:** divide el código fuente en tokens (palabras reservadas, identificadores, operadores, delimitadores, constantes) y reporta errores léxicos, como un carácter no reconocido por el lenguaje.
2. **Análisis sintáctico:** verifica que la secuencia de tokens siga la gramática de `Piper` y construye el árbol sintáctico (AST); reporta errores sintácticos, como un `.` faltante al final de una instrucción o un bloque sin cerrar.
3. **Análisis semántico:** verifica tipos, ámbito de variables y, en programas con múltiples archivos, resuelve las referencias declaradas con `bring ... from` contra los símbolos que exportan los demás módulos; aquí se reportan tanto errores semánticos como los errores de referencias entre archivos.
4. **Generación de código objeto:** por cada archivo se genera un `.obj` con el código traducido más su tabla de símbolos (los que exporta y los que aún tiene pendientes de resolver).
5. **Enlazado:** combina los `.obj` de todos los archivos del programa, resuelve las referencias pendientes contra la tabla global de símbolos y produce el binario ejecutable final.

Los errores que involucran referencias entre archivos ocurren en las fases 3 y 5 (resolución semántica y enlazado), ya que es ahí donde el compilador intenta emparejar cada símbolo importado con `bring` contra su definición real. Los casos son:

5.4.1. Símbolo importado no encontrado

```
main.rwt:2:1: error: no se encontró el símbolo 'derive_key' en el
archivo 'keys.rwt'
```

Recuperación: no se aborta de inmediato; se sigue compilando el resto del archivo para reportar más errores en un solo pase, pero la compilación se marca como fallida y no se genera binario.

5.4.2. Conflicto de nombres entre dos módulos importados

```
main.rwt:3:1: error: conflicto de nombres: 'sumar' fue importado  
tanto desde 'math.rwt' como desde 'utils.rwt'
```

Recuperación: error fatal, se detiene el enlazado de ese archivo; se debe renombrar el símbolo en uno de los módulos de origen o usar un alias explícito en el **bring**.

5.4.3. Importación circular (A importa B y B importa A)

```
error: importación circular detectada: 'a.rwt' -> 'b.rwt' -> 'a.rwt'
```

Recuperación: se detecta llevando una pila de archivos en proceso de compilación; si un archivo reaparece en esa pila, se reporta el ciclo completo y se detiene la compilación de todos los archivos involucrados.

6. Ejemplos

6.1. Multiplicación de dos matrices que ingresan por parámetro utilizando recursividad de cola

Versión de Piper

```
dev@compiler: ~/sketch.src

1  ~~ Ejemplo1
2
3  routine int multiplicar_paso[
4      int|2 & 3| MatrizA, int|3 & 2| MatrizB, int|2 & 2| MatrizC,
5      int i, int j, int k, int acumulado
6  ] {
7      ~~ Todas las filas están completas.
8      when [i = 2] {
9          output 0.
10     }
11
12     ~~ Avanzamos a la siguiente fila.
13     when [j = 2] {
14         output multiplicar_paso[A, B, C, i + 1, 0, 0, 0].
15     }
16
17     ~~ Guardamos la celda y avanzamos de columna.
18     when [k = 3] {
19         C|i & j| store acumulado.
20         output multiplicar_paso[A, B, C, i, j + 1, 0, 0].
21     }
22
23     ~~ Acumulamos un término y continuamos.
24     output multiplicar_paso[
25         A, B, C, i, j, k + 1, acumulado + A|i & k| * B|k & j|
26     ].
27 }
28
29 routine int multiplicar[
30     int|2 & 3| A, int|3 & 2| B, int|2 & 2| C
31 ] {
32     output multiplicar_paso[A, B, C, 0, 0, 0, 0].
33 }
34
35 routine int main[] {
36     create int|2 & 3| A.
37     A store [[1, 2, 3], [4, 5, 6]].
38
39     create int|3 & 2| B.
40     B store [[7, 8], [9, 10], [11, 12]].
41
42     create int|2 & 2| C.
43     C store [[0, 0], [0, 0]].
44
45     create int estado.
46     estado store multiplicar[A, B, C].
47
48     output estado.
49 }
50
```

Versión de Python

```
dev@compiler: ~/main.py

def multiplicar_paso(A, B, C, i, j, k, acumulado):
    # Ya terminamos todas las filas.
    if i == 2:
        return 0

    # Terminamos esta fila: avanzamos a la siguiente.
    if j == 2:
        return multiplicar_paso(A, B, C, i + 1, 0, 0, 0)

    # Terminamos una celda: guardamos y avanzamos de columna.
    if k == 3:
        C[i][j] = acumulado
        return multiplicar_paso(A, B, C, i, j + 1, 0, 0)

    # Acumulamos otro término del producto punto.
    return multiplicar_paso(
        A, B, C,
        i, j, k + 1,
        acumulado + A[i][k] * B[k][j]
    )

def multiplicar(A, B, C):
    return multiplicar_paso(A, B, C, 0, 0, 0, 0)
```

6.2. Implementación de manera iterativa del Merge Sort

Versión de Piper

```

1  dev@compiler: ~/sketch.src
2
3  ``Ejemplo2: Merge Sort
4
5  routine int merge[
6      int|7| values, int|7| buffer,
7      int start, int middle, int end
8  ] {
9
10     create int left.
11     left store start.
12
13     create int right.
14     right store middle.
15
16     create int position.
17     position store start.
18
19     alwhen [left < middle && right < end] {
20         when [values|left| <= values|right|] {
21             buffer|position| store values|left|.
22             left store left + 1.
23         }
24         fallback {
25             buffer|position| store values|right|.
26             right store right + 1.
27         }
28
29         position store position + 1.
30     }
31
32     alwhen [left < middle] {
33         buffer|position| store values|left|.
34         left store left + 1.
35         position store position + 1.
36     }
37
38     alwhen [right < end] {
39         buffer|position| store values|right|.
40         right store right + 1.
41         position store position + 1.
42     }
43
44     output 0.
45 }
46
47 routine int merge_sort[int|7| values] {
48     create int|7| buffer.
49     buffer store [0, 0, 0, 0, 0, 0, 0].
50
51     create int width.
52     width store 1.
53
54     create int start.
55     create int middle.
56     create int end.
57     create int position.
58     create int status.
59
60     alwhen [width < 7] {
61         start store 0.
62
63         alwhen [start < 7] {
64             middle store start + width.

```

```

62         end store middle + width.
63
64         when [middle > 7] {
65             middle store 7.
66         }
67
68         when [end > 7] {
69             end store 7.
70         }
71
72         status store merge[values, buffer, start, middle, end].
73         start store end.
74     }
75
76     position store 0.
77
78     alwhen [position < 7] {
79         values[position] store buffer[position].
80         position store position + 1.
81     }
82
83     width store width + width.
84 }
85
86 output 0.
87 }
88
89 routine int main[] {
90     create int[7] values.
91     values store [38, 27, 43, 3, 9, 82, 10].
92
93     create int status.
94     status store merge_sort[values].
95
96     create int position.
97     position store 0.
98
99     alwhen [position < 7] {
100         position store position + 1.
101     }
102
103     output status.
104 }

```

Versión de Python

```
dev@compiler: ~/main.py

1 def merge(values, buffer, start, middle, end):
2     left = start
3     right = middle
4     position = start
5
6     while left < middle and right < end:
7         if values[left] <= values[right]:
8             buffer[position] = values[left]
9             left = left + 1
10        else:
11            buffer[position] = values[right]
12            right = right + 1
13
14        position = position + 1
15
16    while left < middle:
17        buffer[position] = values[left]
18        left = left + 1
19        position = position + 1
20
21    while right < end:
22        buffer[position] = values[right]
23        right = right + 1
24        position = position + 1
25
26    return 0
27
28
29 def merge_sort(values):
30     buffer = [0, 0, 0, 0, 0, 0, 0, 0]
31     width = 1
32
33     while width < 7:
34         start = 0
35
36         while start < 7:
37             middle = start + width
38             end = middle + width
39
40             if middle > 7:
41                 middle = 7
42
43             if end > 7:
44                 end = 7
45
46             status = merge(values, buffer, start, middle, end)
47             start = end
48
49             position = 0
50
51             while position < 7:
52                 values[position] = buffer[position]
53                 position = position + 1
54
55             width = width + width
56
57     return 0
58
59
60 def main():
61     values = [38, 27, 43, 3, 9, 82, 10]
```

```
62     status = merge_sort(values)
63
64     position = 0
65
66     while position < 7:
67         print(values[position])
68         position = position + 1
69
70     return status
71
72
73 if __name__ == "__main__":
74     raise SystemExit(main())
```

6.3. Un algoritmo propuesto por cada grupo donde se evidencie el uso de for, if, llamadas a métodos y uso de expresiones en parámetros, además de la importación de dos archivos.

Versión de Piper

dev@compiler: ~/keys.rwt

```
1 routine int derive_key[int value] {  
2   when [value >= 256] {  
3     output value - 256.  
4   }  
5  
6   output value.  
7 }
```

dev@compiler: ~/cipher.rwt

```
1 routine int transform[int value, int key] {  
2   output value xor key.  
3 }
```

dev@compiler: ~/main.rwt

```
1 bring derive_key from "keys.rwt".  
2 bring transform from "cipher.rwt".  
3  
4 routine bool main[] {  
5   create int|4| message.  
6   message store [65, 66, 67, 68].  
7  
8   create int seed.  
9   seed store 254.  
10  
11   create int key.  
12   create int encrypted.  
13   create int restored.  
14  
15   loop[position, 0 -> 4] {  
16     key store derive_key[seed + position].  
17  
18     encrypted store transform[message[position], key].  
19     restored store transform[encrypted, key].  
20  
21     when [restored = message[position]] {  
22       output true.  
23     }  
24     fallback {  
25       return false.  
26     }  
27   }  
28 }
```

Versión en Python

```
dev@compiler: ~/keys.py
```

```
def derive_key(value):  
    if value >= 256:  
        return value - 256  
  
    return value
```

```
dev@compiler: ~/cipher.py
```

```
def transform(value, key):  
    return value ^ key
```

```
dev@compiler: ~/main.py
```

```
from keys import derive_key  
from cipher import transform  
  
def main():  
    message = [65, 66, 67, 68]  
    seed = 254  
  
    for position in range(0, 4):  
        key = derive_key(seed + position)  
  
        encrypted = transform(message[position], key)  
        restored = transform(encrypted, key)  
  
        print(encrypted)  
  
        if restored == message[position]:  
            print("OK")  
        else:  
            print("Mismatch")  
  
if __name__ == "__main__":  
    main()
```

Conclusión

El diseño sintáctico presentado en este primer avance sienta las bases léxicas y gramaticales sobre las cuales se construirá la fase de análisis del compilador. La estructuración clara de tokens, la delimitación explícita de bloques y la inclusión de construcciones de control, permiten definir un lenguaje de programación coherente, con la capacidad de manejar desde estructuras de datos vectoriales hasta rutinas recursivas. Este trabajo inicial no sólo valida la viabilidad gramatical de la sintaxis propuesta, sino que también sirve como el modelo directo para la implementación posterior del analizador léxico y del analizador sintáctico.